

🐼 Pandas Basic to Advance Guide

- Created by:----- Harsh Choudhary

In [104...



What is pandas 🐼

Pandas is a powerful open-source Python library used for data analysis and data manipulation. It provides easy-to-use data structures and functions designed to make working with structured data (like tables, spreadsheets, and databases) fast and simple.

- Key Features of Pandas

Data Structures:

Series → One-dimensional labeled array (like a column in Excel).

DataFrame → Two-dimensional labeled data structure (like an Excel sheet or SQL table).

Data Handling:

Import and export data from CSV, Excel, JSON, SQL, HTML, etc.

Handle missing data, filter, sort, and group data easily.

Data Analysis:

Perform aggregation, merging, joining, pivoting, and reshaping of data.

Apply mathematical, statistical, and custom functions to datasets.

Integration:

Works seamlessly with NumPy, Matplotlib, Scikit-learn, and other data science libraries.

- Why Pandas is Popular

Simplifies data cleaning and transformation.

Handles large datasets efficiently.

Integrates well with machine learning and data visualization tools.

In [3]: `import pandas as pd`

Read Data

- `pd.read_csv("bank-marketing.csv")`

This function reads data from a CSV file (Comma-Separated Values).

"bank-marketing.csv" is the filename (it must exist in your current working directory, or you must provide the full path).

- What happens:

Pandas opens the file.

Reads the data row by row.

Automatically converts it into a DataFrame — a table-like structure with rows and columns.

- df
- df is just a variable name. It stores the DataFrame created from the CSV.
- Typing df (in a Jupyter Notebook or Python console) displays the contents of the DataFrame.
- Summary:

Code Part	Meaning
pd	Alias for pandas
read_csv()	Reads a CSV file
"bank-marketing.csv"	Name of your dataset file
df	Stores the loaded data as a pandas DataFrame
df (on its own)	Displays the table-like dataset

In [4]:

```
df = pd.read_csv("bank-marketing.csv")
df
```

Out[4]:

	age	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	contact	day	month	duration	campaign	...
0	58	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	unknown	5	may	261	1	...
1	44	4	Y	technician	60000	single	secondary	single-secondary	yes	no	...	unknown	5	may	151	1	...
2	33	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	unknown	5	may	76	1	...
3	47	4	Y	blue-collar	20000	married	unknown	married-unknown	no	no	...	unknown	5	may	92	1	...
4	33	3	Y	unknown	0	single	unknown	single-unknown	no	no	...	unknown	5	may	198	1	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51	5	Y	technician	60000	married	tertiary	married-tertiary	yes	no	...	cellular	17	nov	977	3	...
45207	71	7	N	retired	55000	divorced	primary	divorced-primary	yes	no	...	cellular	17	nov	456	2	...
45208	72	7	N	retired	55000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	1127	5	...
45209	57	5	Y	blue-collar	20000	married	secondary	married-secondary	yes	no	...	telephone	17	nov	508	4	...
45210	37	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	361	2	...

45211 rows × 23 columns

◀

▶

Pandas DataFrame manually

- 1. data = {...}

You created a Python dictionary named data.

A dictionary stores data as key–value pairs.

Here:

"Name" is one key with a list of names as values.

"Age" is another key with a list of ages as values.

- pd.DataFrame(data)

This line converts the dictionary into a Pandas DataFrame (a table-like structure).

--Each key becomes a column name. --Each list item becomes a row entry.

Code	Meaning
data = {...}	Create a dictionary of data

Code	Meaning
pd.DataFrame(data)	Convert it into a table (DataFrame)
df	Display the DataFrame

```
In [5]: data = {
        "Name":["Ram", "Shyam", "Rahul", "Anuj"],
        "Age"  : [25, 20, 22, 21]
      }

df= pd.DataFrame(data)
df
```

Out[5]:

	Name	Age
0	Ram	25
1	Shyam	20
2	Rahul	22
3	Anuj	21

save your DataFrame to a CSV file

1. df.to_csv(...)

This exports (saves) your DataFrame df into a CSV file format.

2. "output.csv"

The name of the file that will be created in your working directory.

You can use a full path like "C:/Users/Harsh/Desktop/output.csv" if you want to save it in a specific location.

3. index=False

Tells pandas not to write the row numbers (the index column) into the CSV file.

If you set index=True, pandas will include those row numbers as an extra column.

Part	Meaning
df	Your pandas DataFrame variable
.to_csv()	Pandas function to export (save) the DataFrame
"filename.csv"	The name of the file you want to create
index=False	Prevents pandas from writing the row numbers (index) into the CSV file

Summary Table for All Type of file Save

File Type	Function	Example	Use Case
.csv	to_csv()	df.to_csv("data.csv")	Common, readable format
.xlsx	to_excel()	df.to_excel("data.xlsx")	Excel sharing
.json	to_json()	df.to_json("data.json")	Web / APIs
.html	to_html()	df.to_html("data.html")	Display on websites
.txt	to_csv(sep="\t")	df.to_csv("data.txt")	Simple text logs
.db / .sqlite	to_sql()	df.to_sql("table", engine)	Database storage
.pkl	to_pickle()	df.to_pickle("data.pkl")	Fast Python storage
.gz	to_csv(compression="gzip")	df.to_csv("data.csv.gz")	Compressed CSV

```
In [11]: # Save in CSV
df.to_csv("output.csv", index=False)

# Save in Excel
df.to_excel("data.xlsx", index=False, sheet_name="Sheet1")

# Save to JSON
df.to_json("data.json", orient="records", indent=4)

# Save to HTML
df.to_html("data.html", index=False)

# Save to TXT
```

```
df.to_csv("data.txt", sep="\t", index=False)

# # Save to Pickle
df.to_pickle("data.pkl")

# Save to Compressed CSV
df.to_csv("data.csv.gz", index=False, compression="gzip")
```

```
In [5]: df = pd.read_csv("bank-marketing.csv")
df
```

Out[5]:

	age	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	contact	day	month	duration	campaign	...
0	58	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	unknown	5	may	261	1	...
1	44	4	Y	technician	60000	single	secondary	single-secondary	yes	no	...	unknown	5	may	151	1	...
2	33	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	unknown	5	may	76	1	...
3	47	4	Y	blue-collar	20000	married	unknown	married-unknown	no	no	...	unknown	5	may	92	1	...
4	33	3	Y	unknown	0	single	unknown	single-unknown	no	no	...	unknown	5	may	198	1	...
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51	5	Y	technician	60000	married	tertiary	married-tertiary	yes	no	...	cellular	17	nov	977	3	...
45207	71	7	N	retired	55000	divorced	primary	divorced-primary	yes	no	...	cellular	17	nov	456	2	...
45208	72	7	N	retired	55000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	1127	5	...
45209	57	5	Y	blue-collar	20000	married	secondary	married-secondary	yes	no	...	telephone	17	nov	508	4	...
45210	37	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	361	2	...

45211 rows × 23 columns



df.head()

Syntax: df.head(n)

Meaning:

Shows the first n rows of your DataFrame.

If you don't specify n, it shows the first 5 rows by default.

df.tail()

Syntax: df.tail(n)

Meaning:

Shows the last n rows of your DataFrame.

If you don't specify n, it shows the last 5 rows.

Function	Purpose	Default Rows	Example
df.head()	Shows the first few rows	5	df.head(3)
df.tail()	Shows the last few rows	5	df.tail(2)

```
In [18]: df.head()
```

Out[18]:

	age	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	contact	day	month	duration	campaign	pdays
0	58	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	unknown	5	may	261	1	-1
1	44	4	Y	technician	60000	single	secondary	single-secondary	yes	no	...	unknown	5	may	151	1	-1
2	33	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	unknown	5	may	76	1	-1
3	47	4	Y	blue-collar	20000	married	unknown	married-unknown	no	no	...	unknown	5	may	92	1	-1
4	33	3	Y	unknown	0	single	unknown	single-unknown	no	no	...	unknown	5	may	198	1	-1

5 rows × 23 columns

In [19]: df.tail()

Out[19]:

	age	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	contact	day	month	duration	campaign	pdays
45206	51	5	Y	technician	60000	married	tertiary	married-tertiary	yes	no	...	cellular	17	nov	977		3
45207	71	7	N	retired	55000	divorced	primary	divorced-primary	yes	no	...	cellular	17	nov	456		2
45208	72	7	N	retired	55000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	1127		5
45209	57	5	Y	blue-collar	20000	married	secondary	married-secondary	yes	no	...	telephone	17	nov	508		4
45210	37	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	361		2

5 rows × 23 columns

Summary Table

Function	Purpose
df.info()	Overview of structure, columns, data types, non-null counts
df.dtypes	Shows only data types
df.shape	Shows number of rows & columns
df.describe()	Shows statistical summary (for numeric data)

df.info()

- What It Does:

Displays a summary of the DataFrame, including:

Total number of rows and columns

Column names

Data types (int64, float64, object, etc.)

Number of non-null (non-missing) values in each column

Memory usage

In [20]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    45211 non-null  int64
1   age group              45211 non-null  int64
2   eligible               45211 non-null  object
3   job                    45211 non-null  object
4   salary                 45211 non-null  int64
5   marital                45211 non-null  object
6   education              45211 non-null  object
7   marital-education      45211 non-null  object
8   targeted               45211 non-null  object
9   default                45211 non-null  object
10  balance                 45211 non-null  int64
11  housing                 45211 non-null  object
12  loan                    45211 non-null  object
13  contact                 45211 non-null  object
14  day                     45211 non-null  int64
15  month                  45211 non-null  object
16  duration                45211 non-null  int64
17  campaign                45211 non-null  int64
18  pdays                  45211 non-null  int64
19  previous                45211 non-null  int64
20  poutcome                45211 non-null  object
21  y                       45211 non-null  object
22  response                45211 non-null  int64
dtypes: int64(10), object(13)
memory usage: 7.9+ MB
```

df.dtypes

- What It Does:

Returns a pandas Series with column names as the index and data types as the values.

Helps you understand what kind of data is stored in each column:

int64 → integer numbers

float64 → decimal numbers

object → strings or mixed types

bool → Boolean (True/False)

datetime64 → date and time

```
In [22]: df.dtypes

Out[22]: age                    int64
age group              int64
eligible               object
job                    object
salary                 int64
marital                object
education              object
marital-education      object
targeted               object
default                object
balance                 int64
housing                 object
loan                    object
contact                 object
day                     int64
month                  object
duration                int64
campaign                int64
pdays                  int64
previous                int64
poutcome                object
y                       object
response                int64
dtype: object
```

df.shape

- What It Does:

Returns a tuple (number_of_rows, number_of_columns)

Helps you quickly know how big your dataset is.


```
In [23]: df.shape
```

```
Out[23]: (45211, 23)
```

df.describe()

- What It Does:

Returns summary statistics of numeric columns by default.

Includes

Statistic	Meaning
count	Number of non-missing values
mean	Average value
std	Standard deviation (spread of data)
min	Minimum value
25% (Q1)	25th percentile (lower quartile)
50% (median)	50th percentile (middle value)
75% (Q3)	75th percentile (upper quartile)
max	Maximum value

```
In [24]: df.describe()
```

	age	age group	salary	balance	day	duration	campaign	pdays	previous	response
count	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000	45211.000000
mean	40.936210	3.645861	57006.171065	1362.272058	15.806419	258.163080	2.763841	40.197828	0.580323	0.116985
std	10.618762	1.083271	32085.718415	3044.765829	8.322476	257.527812	3.098021	100.128746	2.303441	0.321406
min	18.000000	1.000000	0.000000	-8019.000000	1.000000	0.000000	1.000000	-1.000000	0.000000	0.000000
25%	33.000000	3.000000	20000.000000	72.000000	8.000000	103.000000	1.000000	-1.000000	0.000000	0.000000
50%	39.000000	3.000000	60000.000000	448.000000	16.000000	180.000000	2.000000	-1.000000	0.000000	0.000000
75%	48.000000	4.000000	70000.000000	1428.000000	21.000000	319.000000	3.000000	-1.000000	0.000000	0.000000
max	95.000000	9.000000	120000.000000	102127.000000	31.000000	4918.000000	63.000000	871.000000	275.000000	1.000000

df.columns

- Explanation:

df.columns returns a pandas Index object containing all column names.

You don’t use parentheses () because it is an attribute, not a method.

If you want a list of column names, you can do:

```
In [26]: df.columns
```

```
Out[26]: Index(['age', 'age group', 'eligible', 'job', 'salary', 'marital', 'education',
              'marital-education', 'targeted', 'default', 'balance', 'housing',
              'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays',
              'previous', 'poutcome', 'y', 'response'],
              dtype='object')
```

df.iloc[0]

- What iloc Is

iloc stands for integer location.

It is used to select rows or columns by their integer index positions (starting at 0).

Works with numbers only, not column names.

Summary

Code	Meaning
<code>df.iloc[0]</code>	First row as Series
<code>df.iloc[0:2]</code>	First two rows as DataFrame
<code>df.iloc[0, 0]</code>	First row, first column value
<code>df.iloc[-1]</code>	Last row

```
In [28]: df.iloc[0]
```

```
Out[28]: age                58
age group                5
eligible                 Y
job                management
salary             100000
marital                married
education            tertiary
marital-education    married-tertiary
targeted              yes
default               no
balance              2143
housing              yes
loan                 no
contact            unknown
day                  5
month                may
duration             261
campaign              1
pdays              -1
previous              0
poutcome            unknown
y                    no
response              0
Name: 0, dtype: object
```

```
In [12]: df.iloc[0:2]
```

Out[12]:

	age	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	contact	day	month	duration	campaign	pdays
0	58	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	unknown	5	may	261	1	-1
1	44	4	Y	technician	60000	single	secondary	single-secondary	yes	no	...	unknown	5	may	151	1	-1

2 rows × 23 columns

```
In [15]: df.iloc[2,5]
```

Out[15]: 'married'

```
In [16]: df.iloc[-1]
```

```
Out[16]: age                37
age group                3
eligible                 Y
job                entrepreneur
salary             120000
marital                married
education            secondary
marital-education    married-secondary
targeted              yes
default               no
balance              2971
housing              no
loan                 no
contact            cellular
day                  17
month                nov
duration             361
campaign              2
pdays              188
previous              11
poutcome            other
y                    no
response              0
Name: 45210, dtype: object
```

Filtering

Summary Table:


```
| Task | Code Example | | | ----- | ----- | ----- | | Filter numeric | df[df["Age"] > 21] | |
| | Multiple conditions AND | df[(df["Age"] > 21) & (df["City"]=="Delhi")] | | | Multiple conditions OR | df[(df["Age"] > 21)
(df["City"]=="Mumbai")] | | Filter strings exact | df[df["Name"]=="Anuj"] | | | Filter strings contains | df[df["Name"].str.contains("a")] | | |
Filter multiple values | df[df["City"].isin(["Delhi","Jaipur"])] | | | Exclude values | df[~df["City"].isin(["Delhi"])] | |
```

- Step-by-Step Explanation 1 df["age"] > 21

This creates a Boolean Series (True/False) for each row:

```
2 df[ ... ]
```

Pandas uses this Boolean Series to select only rows where the condition is True.

Rows where the condition is False are excluded.

```
3 filtered_df
```

Stores the new DataFrame containing only the rows where age > 21.

```
4 print(filtered_df)
```

Displays the filtered rows.

```
In [30]: filtered_df = df[df["age"] > 21]
print(filtered_df)

   age  age group  eligible   job  salary  marital  education \
0    58         5         Y  management  100000  married  tertiary
1    44         4         Y  technician   60000   single  secondary
2    33         3         Y  entrepreneur  120000  married  secondary
3    47         4         Y  blue-collar   20000  married   unknown
4    33         3         Y    unknown         0   single   unknown
...  ...      ...      ...      ...      ...      ...      ...
45206  51         5         Y  technician   60000  married  tertiary
45207  71         7         N    retired   55000  divorced   primary
45208  72         7         N    retired   55000  married  secondary
45209  57         5         Y  blue-collar   20000  married  secondary
45210  37         3         Y  entrepreneur  120000  married  secondary

   marital-education  targeted  default  ...  contact  day  month  duration \
0    married-tertiary        yes      no  ...  unknown    5    may       261
1    single-secondary        yes      no  ...  unknown    5    may       151
2    married-secondary        yes      no  ...  unknown    5    may        76
3    married-unknown         no      no  ...  unknown    5    may        92
4    single-unknown         no      no  ...  unknown    5    may       198
...      ...      ...      ...      ...      ...      ..      ...      ...
45206  married-tertiary        yes      no  ...  cellular   17   nov       977
45207  divorced-primary        yes      no  ...  cellular   17   nov       456
45208  married-secondary        yes      no  ...  cellular   17   nov      1127
45209  married-secondary        yes      no  ...  telephone  17   nov       508
45210  married-secondary        yes      no  ...  cellular   17   nov       361

   campaign  pdays  previous  poutcome   y  response
0          1     -1         0   unknown  no         0
1          1     -1         0   unknown  no         0
2          1     -1         0   unknown  no         0
3          1     -1         0   unknown  no         0
4          1     -1         0   unknown  no         0
...      ...      ...      ...      ...      ...      ...
45206      3     -1         0   unknown  yes         1
45207      2     -1         0   unknown  yes         1
45208      5    184         3   success  yes         1
45209      4     -1         0   unknown  no         0
45210      2    188        11    other   no         0

[45035 rows x 23 columns]
```

```
In [8]: df[["job","salary"]]
```

Out[8]:

	job	salary
0	management	100000
1	technician	60000
2	entrepreneur	120000
3	blue-collar	20000
4	unknown	0
...	...	...
45206	technician	60000
45207	retired	55000
45208	retired	55000
45209	blue-collar	20000
45210	entrepreneur	120000

45211 rows × 2 columns

In [11]:

df[(df["age"] > 21) & (df["salary"]>80000)]

Out[11]:

	age	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	contact	day	month	duration	campaign	p
0	58	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	unknown	5	may	261	1	
2	33	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	unknown	5	may	76	1	
5	35	3	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	unknown	5	may	139	1	
6	28	2	Y	management	100000	single	tertiary	single-tertiary	no	no	...	unknown	5	may	217	1	
7	42	4	Y	entrepreneur	120000	divorced	tertiary	divorced-tertiary	no	yes	...	unknown	5	may	380	1	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45194	59	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	cellular	16	nov	162	2	
45197	36	3	Y	management	100000	single	secondary	single-secondary	yes	no	...	cellular	16	nov	270	1	
45198	37	3	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	cellular	16	nov	333	2	
45201	53	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	cellular	17	nov	226	1	
45210	37	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	361	2	

10942 rows × 23 columns



In [17]:

df[(df["age"] > 21)|(df["job"]=="management")]

Out[17]:

	age	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	contact	day	month	duration	campaign	pdays
0	58	5	Y	management	100000	married	tertiary	married-tertiary	yes	no	...	unknown	5	may	261	1	-1
1	44	4	Y	technician	60000	single	secondary	single-secondary	yes	no	...	unknown	5	may	151	1	-1
2	33	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	unknown	5	may	76	1	-1
3	47	4	Y	blue-collar	20000	married	unknown	married-unknown	no	no	...	unknown	5	may	92	1	-1
4	33	3	Y	unknown	0	single	unknown	single-unknown	no	no	...	unknown	5	may	198	1	-1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
45206	51	5	Y	technician	60000	married	tertiary	married-tertiary	yes	no	...	cellular	17	nov	977	3	-1
45207	71	7	N	retired	55000	divorced	primary	divorced-primary	yes	no	...	cellular	17	nov	456	2	-1
45208	72	7	N	retired	55000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	1127	5	-1
45209	57	5	Y	blue-collar	20000	married	secondary	married-secondary	yes	no	...	telephone	17	nov	508	4	-1
45210	37	3	Y	entrepreneur	120000	married	secondary	married-secondary	yes	no	...	cellular	17	nov	361	2	-1

45037 rows × 23 columns



Advanced Pandas for Data Science & Analysis

Modifying Data

Task	Example
Modify entire column	<code>df["Age"] = df["Age"] + 1</code>
Modify with condition	<code>df.loc[df["Age"]>22, "City"] = "Unknown"</code>
Create new column	<code>df["Senior"] = df["Age"]>22</code>
Modify single cell	<code>df.loc[1, "City"]="Pune"</code>
Rename columns	<code>df.rename(columns={"Age": "Years"}, inplace=True)</code>
Apply function	<code>df["Name"] = df["Name"].apply(str.upper)</code>
Replace values	<code>df["City"].replace({"Delhi": "New Delhi"}, inplace=True)</code>
Fill missing values	<code>df["City"].fillna("Unknown", inplace=True)</code>

```
In [22]: df["age"] = df["age"] + 1
df.loc[df["age"]>22, "eligible"] = "N"
```

```
In [35]: df.head()
```

	Years	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	day	month	duration	campaign	pdays	previous
0	62	5	N	MANAGEMENT	56600	married	tertiary	married-tertiary	yes	no	...	5	may	261	1	-1	0
1	48	4	N	TECHNICIAN	60000	single	secondary	single-secondary	yes	no	...	5	may	151	1	-1	0
2	37	3	N	ENTREPRENEUR	120000	married	secondary	married-secondary	yes	no	...	5	may	76	1	-1	0
3	51	4	N	BLUE-COLLAR	20000	married	unknown	married-unknown	no	no	...	5	may	92	1	-1	0
4	37	3	N	UNKNOWN	0	single	unknown	single-unknown	no	no	...	5	may	198	1	-1	0

5 rows × 24 columns




```
In [34]: df["job"] = df["job"].apply(str.upper)
df.head()
```

Out[34]:

	Years	age group	eligible	job	salary	marital	education	marital-education	targeted	default	...	day	month	duration	campaign	pdays	previc
0	62	5	N	MANAGEMENT	56600	married	tertiary	married-tertiary	yes	no	...	5	may	261	1	-1	
1	48	4	N	TECHNICIAN	60000	single	secondary	single-secondary	yes	no	...	5	may	151	1	-1	
2	37	3	N	ENTREPRENEUR	120000	married	secondary	married-secondary	yes	no	...	5	may	76	1	-1	
3	51	4	N	BLUE-COLLAR	20000	married	unknown	married-unknown	no	no	...	5	may	92	1	-1	
4	37	3	N	UNKNOWN	0	single	unknown	single-unknown	no	no	...	5	may	198	1	-1	

5 rows × 24 columns



Summary Table Remove Data

Remove	Code
Column	<code>df.drop(columns=["col"])</code> or <code>del df["col"]</code>
Row by index	<code>df.drop(index=0)</code>
Row by condition	<code>df = df[df["Age"] >= 22]</code>
Duplicates	<code>df.drop_duplicates()</code>
Missing values	<code>df.dropna()</code>
All rows	<code>df.drop(df.index, inplace=True)</code>

```
In [40]: del df["y"]
df.head(3)
```

Out[40]:

	Years	age group	eligible	job	salary	marital	education	marital-education	default	balance	...	contact	day	month	duration	campaign	pda
0	62	5	N	MANAGEMENT	56600	married	tertiary	married-tertiary	no	2143	...	unknown	5	may	261	1	
1	48	4	N	TECHNICIAN	60000	single	secondary	single-secondary	no	29	...	unknown	5	may	151	1	
2	37	3	N	ENTREPRENEUR	120000	married	secondary	married-secondary	no	2	...	unknown	5	may	76	1	

3 rows × 22 columns



```
In [42]: df.head().dropna()
```

Out[42]:

	Years	age group	eligible	job	salary	marital	education	marital-education	default	balance	...	contact	day	month	duration	campaign	pda
0	62	5	N	MANAGEMENT	56600	married	tertiary	married-tertiary	no	2143	...	unknown	5	may	261	1	
1	48	4	N	TECHNICIAN	60000	single	secondary	single-secondary	no	29	...	unknown	5	may	151	1	
2	37	3	N	ENTREPRENEUR	120000	married	secondary	married-secondary	no	2	...	unknown	5	may	76	1	
3	51	4	N	BLUE-COLLAR	20000	married	unknown	married-unknown	no	1506	...	unknown	5	may	92	1	
4	37	3	N	UNKNOWN	0	single	unknown	single-unknown	no	1	...	unknown	5	may	198	1	

5 rows × 22 columns



```
In [43]: df.drop_duplicates()
df.head(3)
```

Out[43]:

	Years	age group	eligible	job	salary	marital	education	marital-education	default	balance	...	contact	day	month	duration	campaign	pda
0	62	5	N	MANAGEMENT	56600	married	tertiary	married-tertiary	no	2143	...	unknown	5	may	261	1	
1	48	4	N	TECHNICIAN	60000	single	secondary	single-secondary	no	29	...	unknown	5	may	151	1	
2	37	3	N	ENTREPRENEUR	120000	married	secondary	married-secondary	no	2	...	unknown	5	may	76	1	

3 rows × 22 columns

In [48]:

df.head().drop(index=0)

Out[48]:

	Years	age group	eligible	job	salary	marital	education	marital-education	default	balance	...	contact	day	month	duration	campaign	pda
1	48	4	N	TECHNICIAN	60000	single	secondary	single-secondary	no	29	...	unknown	5	may	151	1	
2	37	3	N	ENTREPRENEUR	120000	married	secondary	married-secondary	no	2	...	unknown	5	may	76	1	
3	51	4	N	BLUE-COLLAR	20000	married	unknown	married-unknown	no	1506	...	unknown	5	may	92	1	
4	37	3	N	UNKNOWN	0	single	unknown	single-unknown	no	1	...	unknown	5	may	198	1	

4 rows × 22 columns

handling missing data

Task	Code
Check missing	df.isna() / df.isna().sum()
Drop rows with missing	df.dropna()
Drop rows by column	df.dropna(subset=["Age"])
Drop columns	df.dropna(axis=1)
Fill missing with value	df["Age"].fillna(0, inplace=True)
Fill with mean/median	df["Age"].fillna(df["Age"].mean())
Forward/Backward fill	df.fillna(method="ffill")
Interpolation	df["Age"].interpolate()
Replace specific missing codes	df.replace("NA", pd.NA)

df.isnull() → Returns a DataFrame of True/False, where:

- True = value is missing (NaN)
- False = value is present
- .head(3) → Shows only the first 3 rows.

In [53]:

df.isnull().head(3)

Out[53]:

	Years	age group	eligible	job	salary	marital	education	marital-education	default	balance	...	contact	day	month	duration	campaign	pdays	previous
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False

3 rows × 22 columns

df.isnull().sum()

- df.isnull() → Returns a DataFrame of True/False, where True = value is missing (NaN).

- `.sum()` → Sums the True values column-wise, giving the number of missing values in each column.

```
In [55]: df.isnull().sum()

Out[55]: Years      0
age group    0
eligible      0
job           0
salary        0
marital       0
education     0
marital-education  0
default       0
balance       0
housing       0
loan          0
contact       0
day           0
month         0
duration      0
campaign      0
pdays        0
previous      0
poutcome      0
response      0
id            0
dtype: int64
```

df.dropna(axis=1, inplace=True)

Explanation

- `axis=1`

Tells pandas to drop columns.

`axis=0` would drop rows instead.

- `inplace=True`

Modifies the original DataFrame directly.

Without `inplace=True`, it would return a new DataFrame instead.

- Effect:

Any column that contains at least one missing value (NaN) will be removed from the DataFrame.

```
In [57]: df.dropna(axis=1,inplace = True)
df.head()
```

```
Out[57]:
```

	Years	age group	eligible	job	salary	marital	education	marital-education	default	balance	...	contact	day	month	duration	campaign	pda
0	62	5	N	MANAGEMENT	56600	married	tertiary	married-tertiary	no	2143	...	unknown	5	may	261	1	
1	48	4	N	TECHNICIAN	60000	single	secondary	single-secondary	no	29	...	unknown	5	may	151	1	
2	37	3	N	ENTREPRENEUR	120000	married	secondary	married-secondary	no	2	...	unknown	5	may	76	1	
3	51	4	N	BLUE-COLLAR	20000	married	unknown	married-unknown	no	1506	...	unknown	5	may	92	1	
4	37	3	N	UNKNOWN	0	single	unknown	single-unknown	no	1	...	unknown	5	may	198	1	

5 rows × 22 columns

```
In [62]: import pandas as pd

data = {
    "Name": ["Ram", "Shyam", "Rahul", "Anuj", None],
    "Age": [25, 20, 22, None, 23],
    "City": ["Delhi", "Mumbai", None, "Kolkata", "Pune"]
}

df1 = pd.DataFrame(data)
df1
```

Out[62]:

	Name	Age	City
0	Ram	25.0	Delhi
1	Shyam	20.0	Mumbai
2	Rahul	22.0	None
3	Anuj	NaN	Kolkata
4	None	23.0	Pune

In [63]:

```
import pandas as pd

data = {
    "Name": ["Ram", "Shyam", "Rahul", "Anuj", None],
    "Age": [25, 20, 22, None, 23],
    "City": ["Delhi", "Mumbai", None, "Kolkata", "Pune"]
}

df1 = pd.DataFrame(data)

print("Before dropping columns:\n", df1)

df1.dropna(axis=1, inplace=True)

print("\nAfter dropping columns:\n", df1)
```

Before dropping columns:

	Name	Age	City
0	Ram	25.0	Delhi
1	Shyam	20.0	Mumbai
2	Rahul	22.0	None
3	Anuj	NaN	Kolkata
4	None	23.0	Pune

After dropping columns:

Empty DataFrame

Columns: []

Index: [0, 1, 2, 3, 4]

In [64]:

```
import pandas as pd

data = {
    "Name": ["Ram", "Shyam", "Rahul", "Anuj", None],
    "Age": [25, 20, 22, None, 23],
    "City": ["Delhi", "Mumbai", None, "Kolkata", "Pune"]
}

df1 = pd.DataFrame(data)
df1
```

Out[64]:

	Name	Age	City
0	Ram	25.0	Delhi
1	Shyam	20.0	Mumbai
2	Rahul	22.0	None
3	Anuj	NaN	Kolkata
4	None	23.0	Pune

In [67]:

```
df1.fillna(0,inplace=True)
df1
```

Out[67]:

	Name	Age	City
0	Ram	25.0	Delhi
1	Shyam	20.0	Mumbai
2	Rahul	22.0	0
3	Anuj	0.0	Kolkata
4	0	23.0	Pune

df.fillna(method="ffill")

- fillna()

Used to fill missing values (NaN) in the DataFrame.

- method="ffill"

Stands for forward fill.

Fills each missing value with the last non-missing value above it in the same column.

- inplace=True

Modifies the original DataFrame.

Without it, it would return a new DataFrame with filled values.

In [102...

```
import pandas as pd

df2 = pd.DataFrame({
    "A": [1, None, 3, None, 5],
    "B": [None, 2, None, 4, None]
})

# Forward fill
df2.ffill(inplace=True)
print(df2)
```

	A	B
0	1.0	NaN
1	1.0	2.0
2	3.0	2.0
3	3.0	4.0
4	5.0	4.0

df["Age"].fillna(df["Age"].mean(), inplace=True)

- Explanation

df["Age"]

Selects the Age column in the DataFrame.

- df["Age"].mean()

Calculates the average of all non-missing Age values.

- fillna(value)

Replaces all missing values (NaN) in the column with value.

- inplace=True

Updates the original DataFrame instead of creating a new one.

In [72]:

```
import pandas as pd
import numpy as np

# -----
# Sample DataFrame
# -----
data = {
    "Name": ["Ram", "Shyam", "Rahul", "Anuj", None],
    "Age": [25, 20, 22, None, 23],
    "City": ["Delhi", "Mumbai", None, "Kolkata", "Pune"],
    "Salary": [50000, None, 45000, 48000, None]
}

df = pd.DataFrame(data)

print("Original DataFrame:\n", df)

# -----
# 1. Detect Missing Data
# -----
print("\nMissing values (True = missing):\n", df.isnull().head())
print("\nMissing values per column:\n", df.isnull().sum())
print("\nTotal missing values in DataFrame:", df.isnull().sum().sum())

# -----
# 2. Drop Missing Data
# -----
# Drop rows with any missing value
df_rows_dropped = df.dropna(axis=0)
print("\nDataFrame after dropping rows with missing values:\n", df_rows_dropped)

# Drop columns with any missing value
df_cols_dropped = df.dropna(axis=1)
print("\nDataFrame after dropping columns with missing values:\n", df_cols_dropped)

# -----
# 3. Fill Missing Data
# -----
# 3a. Fill numeric columns with mean
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

```
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())

# 3b. Fill string/object columns with mode or custom value
df["Name"] = df["Name"].fillna(df["Name"].mode()[0])
df["City"] = df["City"].fillna("Unknown")

# 3c. Forward fill / Backward fill (future-proof)
df_ffill = df.ffmpeg() # forward fill
df_bfill = df.bfill() # backward fill

print("\nDataFrame after filling missing values (with mean/mode and ffill/bfill):\n", df)

# -----
# 4. Interpolation (numeric columns)
# -----
df["Salary"] = df["Salary"].interpolate(method="linear")
print("\nDataFrame after interpolation on Salary:\n", df)

# -----
# 5. Replace specific codes for missing data
# -----
# Example: replace -1 or "NA" with NaN
df.replace(-1, np.nan, inplace=True)
df.replace("NA", np.nan, inplace=True)

print("\nFinal DataFrame:\n", df)
```

Original DataFrame:

	Name	Age	City	Salary
0	Ram	25.0	Delhi	50000.0
1	Shyam	20.0	Mumbai	NaN
2	Rahul	22.0	None	45000.0
3	Anuj	NaN	Kolkata	48000.0
4	None	23.0	Pune	NaN

Missing values (True = missing):

	Name	Age	City	Salary
0	False	False	False	False
1	False	False	False	True
2	False	False	True	False
3	False	True	False	False
4	True	False	False	True

Missing values per column:

Name	1
Age	1
City	1
Salary	2
dtype:	int64

Total missing values in DataFrame: 5

DataFrame after dropping rows with missing values:

	Name	Age	City	Salary
0	Ram	25.0	Delhi	50000.0

DataFrame after dropping columns with missing values:

Empty DataFrame
Columns: []
Index: [0, 1, 2, 3, 4]

DataFrame after filling missing values (with mean/mode and ffill/bfill):

	Name	Age	City	Salary
0	Ram	25.0	Delhi	50000.000000
1	Shyam	20.0	Mumbai	47666.666667
2	Rahul	22.0	Unknown	45000.000000
3	Anuj	22.5	Kolkata	48000.000000
4	Anuj	23.0	Pune	47666.666667

DataFrame after interpolation on Salary:

	Name	Age	City	Salary
0	Ram	25.0	Delhi	50000.000000
1	Shyam	20.0	Mumbai	47666.666667
2	Rahul	22.0	Unknown	45000.000000
3	Anuj	22.5	Kolkata	48000.000000
4	Anuj	23.0	Pune	47666.666667

Final DataFrame:

	Name	Age	City	Salary
0	Ram	25.0	Delhi	50000.000000
1	Shyam	20.0	Mumbai	47666.666667
2	Rahul	22.0	Unknown	45000.000000
3	Anuj	22.5	Kolkata	48000.000000
4	Anuj	23.0	Pune	47666.666667

Pandas Sorting & Aggregation

- Sort by Index | Function | Description | Example | | ----- | ----- | -----
----- || df.sort_index() | Sorts DataFrame by **row index** (ascending by default) | df.sort_index() || df.sort_index(ascending=False) |

- Sorts by index in **descending order** | `df.sort_index(ascending=False)` || `df.sort_index(axis=1)` | Sort columns alphabetically | `df.sort_index(axis=1)` |
- Sort by Column Values | Function | Description | Example || ----- | ----- | ---
----- || `df.sort_values("Age")` | Sorts DataFrame by the "Age" column (ascending) |
`df.sort_values("Age")` || `df.sort_values("Age", ascending=False)` | Sorts by "Age" in descending order | `df.sort_values("Age", ascending=False)` || `df.sort_values(["City", "Age"])` | Sorts by multiple columns | `df.sort_values(["City", "Age"])` ||
`df.sort_values(by="Salary", na_position="first")` | Places NaN values first | `df.sort_values(by="Salary", na_position="first")` |
- Aggregation Functions | Function | Description | Example || ----- | ----- | ----- || `df.sum()` | Sum of values for each column | `df["Salary"].sum()` || `df.mean()` | Mean (average) | `df["Age"].mean()` || `df.median()` | Median (middle value) | `df["Age"].median()` || `df.min()` / `df.max()` | Minimum / Maximum | `df["Salary"].max()` || `df.count()` | Count of non-null values | `df.count()` || `df.std()` / `df.var()` | Standard deviation / variance | `df["Salary"].std()` || `df.describe()` | Summary of all numeric columns | `df.describe()` |
- Grouping and Aggregation (groupby | Operation | Example | Output || ----- | ----- | -----
----- | ----- | ----- || Group by one column | `df.groupby("Department")["Salary"].mean()` | Average salary per department || Multiple aggregations | `df.groupby("Department")["Salary"].agg(["mean", "max", "min"])` | Several stats at once || Group by multiple columns | `df.groupby(["Department", "Age"])["Salary"].sum()` | Total salary by dept and age || Named aggregations | `df.groupby("Department").agg(Avg_Salary=("Salary", "mean"), Max_Age=("Age", "max"))` | Custom column names || Count per group | `df.groupby("Department").size()` | Row count per group || Reset index | `df.groupby("Department")["Salary"].mean().reset_index()` | Converts back to normal DataFrame |
- Aggregation on the Whole DataFrame | Function | Example | Description || ----- | ----- | -----
----- || `df.agg(["mean", "max", "min"])` | Aggregates all numeric columns | Mean, max, min for all || `df.agg({"Salary": "mean", "Age": "max"})` | Aggregates specific columns | Custom aggregations || `df.agg(np.sum)` | Uses NumPy functions | Sum for each numeric column |

In [78]:

```
df.sort_values(by=["Age", "Salary"], ascending=[True, False], inplace=True)
df
```

Out[78]:

	Name	Age	City	Salary
1	Shyam	20.0	Mumbai	47666.666667
2	Rahul	22.0	Unknown	45000.000000
3	Anuj	22.5	Kolkata	48000.000000
4	Anuj	23.0	Pune	47666.666667
0	Ram	25.0	Delhi	50000.000000

In [84]:

```
a=df["Age"].mean()
b= df["Age"].median()
c= df["Salary"].mean()
d= df["Salary"].median()
e= df["Salary"].max()
print(a )
print(b )
print(c )
print(d )
print(e )
```

22.5
22.5
47666.666666666664
47666.666666666664
50000.0

In [87]:

```
df.agg("sum")
```

Out[87]:

Name	ShyamRahulAnujAnujRam
Age	112.5
City	MumbaiUnknownKolkataPuneDelhi
Salary	238333.333333
dtype:	object

Pandas groupby()

- What is groupby()?

The groupby() function is used to split data into groups, apply a function (like sum, mean, etc.), and combine the results.

Think of it like this:

“Group data by one or more columns and calculate summary statistics.”

Operation	Code Example	Description
Mean per group	<code>df.groupby("Dept")["Salary"].mean()</code>	Average salary

Operation	Code Example	Description
Count rows	<code>df.groupby("Dept").size()</code>	Row count
Multiple stats	<code>df.groupby("Dept")["Salary"].agg(["mean", "max"])</code>	Several aggregations
Different columns	<code>df.groupby("Dept").agg({"Age": "mean", "Salary": "max"})</code>	Custom per column
Rename columns	<code>df.groupby("Dept").agg(Avg=("Salary", "mean"))</code>	Named output
Reset index	<code>.reset_index()</code>	Converts to normal DataFrame
Sort after grouping	<code>.sort_values(ascending=False)</code>	Sort results

```
In [89]: df.groupby("Age")["Salary"].mean()  
df
```

Out[89]:

	Name	Age	City	Salary
1	Shyam	20.0	Mumbai	47666.666667
2	Rahul	22.0	Unknown	45000.000000
3	Anuj	22.5	Kolkata	48000.000000
4	Anuj	23.0	Pune	47666.666667
0	Ram	25.0	Delhi	50000.000000

```
In [90]: df.groupby("Salary").size()
```

Out[90]:

Salary	
45000.000000	1
47666.666667	2
48000.000000	1
50000.000000	1
dtype: int64	

```
In [92]: df.reset_index()  
df
```

Out[92]:

	Name	Age	City	Salary
1	Shyam	20.0	Mumbai	47666.666667
2	Rahul	22.0	Unknown	45000.000000
3	Anuj	22.5	Kolkata	48000.000000
4	Anuj	23.0	Pune	47666.666667
0	Ram	25.0	Delhi	50000.000000

What is Merging & Joining?

In pandas, merging and joining are used to combine two or more DataFrames based on common columns or indices — just like SQL joins (INNER JOIN, LEFT JOIN, etc.).

Method / Function	Description	Key Parameter(s)	Type of Join	Based On	Example Code
<code>pd.merge(df1, df2)</code>	Combines two DataFrames based on one or more common columns (like SQL JOIN)	<code>on</code> , <code>how</code> , <code>left_on</code> , <code>right_on</code>	<code>inner</code> , <code>left</code> , <code>right</code> , <code>outer</code>	Column(s)	<code>pd.merge(df1, df2, on="Dept_ID", how="inner")</code>
<code>df1.merge(df2)</code>	Same as <code>pd.merge()</code> but called as method	<code>on</code> , <code>how</code>	All SQL join types	Column(s)	<code>df1.merge(df2, on="Dept_ID", how="left")</code>
<code>.join()</code>	Joins DataFrames using their index or a key column	<code>on</code> , <code>how</code>	<code>left</code> , <code>right</code> , <code>inner</code> , <code>outer</code>	Index (default)	<code>df1.join(df2, how="inner")</code>
<code>pd.concat()</code>	Combines DataFrames vertically (stack rows) or horizontally (add columns)	<code>axis</code> , <code>ignore_index</code> , <code>keys</code>	N/A	Index or Columns	<code>pd.concat([df1, df2], axis=0)</code>
<code>append()</code> <i>(deprecated)</i>	Used to add rows from one DataFrame to another	—	—	—	<code>df1.append(df2)</code> → <i>(Use <code>concat()</code> instead)</i>
<code>combine_first()</code>	Combines two DataFrames; fills missing values from another	—	—	Index	<code>df1.combine_first(df2)</code>

```
In [93]: pd.merge(df1,df2, on="City", how="inner")
```

Out[93]:

	Name_x	Age_x	City	Name_y	Age_y
0	Ram	25.0	Delhi	Ram	25.0
1	Ram	25.0	Delhi	Ram	25.0
2	Ram	25.0	Delhi	Rahul	22.0
3	Anuj	0.0	Kolkata	Anuj	22.0
4	0	23.0	Pune	Anuj	23.0

In [97]:

```
pd.concat([df1, df2], axis=1)
```

Out[97]:

	Name	Age	City	Name	Age	City
0	Ram	25.0	Delhi	Ram	25.0	Delhi
1	Shyam	20.0	Mumbai	Ram	25.0	Delhi
2	Rahul	22.0	0	Rahul	22.0	Delhi
3	Anuj	0.0	Kolkata	Anuj	22.0	Kolkata
4	0	23.0	Pune	Anuj	23.0	Pune

Join Types Summary

Join Type	Description	Keeps Rows From	Unmatched Rows
Inner Join	Keeps only matching rows between DataFrames	Both	Discarded
Left Join	Keeps all rows from the left DataFrame	Left	Missing on right → NaN
Right Join	Keeps all rows from the right DataFrame	Right	Missing on left → NaN
Outer Join	Keeps all rows from both DataFrames	Both	Missing values → NaN

In [101...

```
import pandas as pd

# Employee Details
df1 = pd.DataFrame({
    "Emp_ID": [101, 102, 103, 104, 105],
    "Name": ["Ram", "Shyam", "Rahul", "Anuj", "Vijay"],
    "Dept_ID": [1, 2, 1, 3, 5]
})

# Department Details
df2 = pd.DataFrame({
    "Dept_ID": [1, 2, 3, 4],
    "Department": ["HR", "Finance", "IT", "Marketing"]
})

print("\nDepartment DataFrame (df2):")
print(df2)

print("Employee DataFrame (df1):")
print(df1)

# INNER JOIN
pd.merge(df1, df2, on="Dept_ID", how="inner")

# LEFT JOIN
pd.merge(df1, df2, on="Dept_ID", how="left")

# RIGHT JOIN
pd.merge(df1, df2, on="Dept_ID", how="right")

# OUTER JOIN
pd.merge(df1, df2, on="Dept_ID", how="outer")

# JOIN BY INDEX
df1.set_index("Emp_ID").join(df2.set_index("Dept_ID"), how="left")

# CONCAT (Vertical or Horizontal)
pd.concat([df1, df2], axis=0) # vertical (rows)
pd.concat([df1, df2], axis=1) # horizontal (columns)
```

```
Department DataFrame (df2):
  Dept_ID Department
0         1         HR
1         2    Finance
2         3         IT
3         4  Marketing
Employee DataFrame (df1):
  Emp_ID  Name  Dept_ID
0     101   Ram        1
1     102 Shyam        2
2     103 Rahul        1
3     104  Anuj        3
4     105 Vijay        5
```

Out[101...

	Emp_ID	Name	Dept_ID	Dept_ID	Department
0	101	Ram	1	1.0	HR
1	102	Shyam	2	2.0	Finance
2	103	Rahul	1	3.0	IT
3	104	Anuj	3	4.0	Marketing
4	105	Vijay	5	NaN	NaN



Pandas Quick Summary

1 Basics:

- pd.DataFrame() → Create table
- pd.read_csv(), pd.read_excel() → Read files
- df.to_csv(), df.to_excel() → Save files

2 Explore Data:

- df.head(), df.tail() → View rows
- df.info(), df.describe(), df.shape, df.dtypes

3 Select & Filter:

- df["Col"], df[["Col1","Col2"]] → Columns
- df.iloc[0], df.loc[] → Rows
- df[df["Col"]>value] → Conditional filter

4 Modify Data:

- df.insert(), df.drop() → Add/remove columns
- df["Col"].fillna(), df.dropna() → Handle missing values

5 Sorting & Aggregation:

- df.sort_values("Col") → Sort
- df["Col"].mean(), .sum(), .agg() → Summary statistics
- df.groupby("Col").agg() → Group & summarize

6 Merge & Join:

- pd.merge(df1, df2, on="Col", how="inner/left/right/outer") → Merge tables
- df1.join(df2) → Join by index
- pd.concat([df1, df2], axis=0/1) → Stack vertically/horizontally

Thankyou Everyone